# Module 2, Lecture 1 — From Ambiguous Request to PRD: Scope Definition

> **Course:** Staff Engineer / Technical Lead Mentorship **Module:** 2 — PRD, scope definition, MVP, user stories, and acceptance criteria **Lecture:** 1 of Module 2 **Case study:** A personal finance app (React Native + FastAPI + PostgreSQL) **Format:** A one-way lecture. I demonstrate everything myself, including the exercise and its review. Nothing in this file waits for your reply. You only need to read, absorb, and, if you wish, try the exercise on your own before reading the answer.

---

## Part 1 — Learning objective

After this lecture you should be able to take a vague request and turn it into a short, testable Product Requirements Document (PRD) section. Specifically, you should be able to:

1. Convert an ambiguous request into a **business outcome** with a measurable success metric.
2. Write **functional requirements** (what the system does) and **non-functional requirements** (how well it does it) that can be tested.
3. Decide **MVP scope** with a defensible reason for every item that is in or out.
4. Write **user stories** and **acceptance criteria** that a developer and a tester would interpret the same way.
5. Record **assumptions, dependencies, and open questions** so that risk is visible.
6. Write **explicit exclusions** (what we are *not* building) so that scope does not grow silently.

I will go in this order: scenario, thought process, a step-by-step demonstration, a sample PRD, documentation advice, trade-offs, an exercise, a review of a typical attempt, a recall test, and a checklist.

---

## Part 2 — Real-world scenario

Let's continue with our freelance client, Meera, from Module 1. Imagine that discovery is done. You had two conversations with her and wrote the notes. This is what you have in your folder (all of it is invented for teaching):

**Meera's confirmed points**

- She tracks money in a notebook today, forgets to write things down, and at month-end has no idea where the money went.
- The first version is for her and about 10 friends.

- No bank connection at first. Launch date is "not fixed," but she told her friends "in a couple of months."
- Android and iOS are both wanted, because her friends use both.

**The messy wish-list that arrived afterwards, by message, over a week**

- Meera: "Can it show nice charts?"
- Meera's friend Rahul: "Auto-import from my bank. Also works offline."
- Meera's friend Anita: "Budgets with warnings when I overspend."
- Rahul's wife: "We want to share one budget."
- Meera: "Dark mode would be cool."
- Meera: "Maybe receipts scanning later?"
- Meera: "It must be super easy and super fast and super secure."

Now think about what is on your desk: a goal, a few confirmed facts, and seven wishes that all arrived with equal urgency. If you open the editor now, you will build whatever was said last or said loudest.

This is the moment where a lead writes a **PRD**. Not because documents are fun, but because someone has to decide, in writing, *what is in, what is out, and how we will know it worked.* Today I'll show how.

---

## Part 3 — The lead's thought process

### 3.1 What a PRD is, and what it is not

A **Product Requirements Document (PRD)** describes **what** the product must do and **why**, for **whom**, and **how success is measured**. It is the agreement between the people who want the product and the people who build it.

What it is *not*:

- It is **not a technical design.** It does not say "use Redux" or "create a `budgets` table." Those are *solutions*, and they belong in the technical design (Module 3). Remember the ladder from Lecture 1: goal, problem, requirement, solution. The PRD lives on the *requirement* level. When a PRD slips into solution language, it removes options from the engineers for no reason.
- It is **not a contract written to trap anyone.** It is a shared understanding. In freelance work it may be *attached* to a contract, which is a good habit, but its purpose is clarity.
- It is **not a novel.** For a small MVP, three to six pages is usually enough. If the PRD is longer than the software is complex, you are probably writing to feel safe rather than to be clear.

- It is **not frozen.** It is a living document with a version number and a change log. We will come to that.

**Who reads it?** The client (to confirm intent), you (to build), any teammate you bring in later, a tester, and *you in three months* when you forget why something was excluded. Write for that last reader. They have no memory of the conversations.

**DBA connection.** Think of the PRD as the *data contract of the whole project.* A table has columns, types, constraints, and the constraints say what is *not* allowed. A good PRD has requirements, measurable limits, and exclusions in the same spirit. Vague PRDs are like tables without constraints: they accept anything, and the garbage shows up later.

### 3.2 Step one: from request to business outcome

Here is the first habit. When you read "Can it show nice charts?", do **not** ask "what chart library?" Ask, "What would the chart help Meera *do* or *decide*?"

A business outcome is a **change in the real world** that matters to the client. It is not a feature.

| Request as heard | Output (what we would build) | Outcome (what changes for people) |
|---|---|---|
| "Nice charts" | Bar and pie charts | Users understand where their money goes without doing math |
| "Budgets with warnings" | A budget screen and alerts | Users find out about overspending *before* month-end, not after |
| "Super easy and fast" | A quick-add screen | Users actually record expenses on the day they happen |
| "Auto-import from bank" | Bank integration | Users do not have to type, so they do not forget |

Look at the last row. "Do not have to type, so they do not forget" is the **real need**. Auto-import is only one way to get there. A fast manual entry plus a daily reminder might reach the same outcome at a tenth of the cost and without handling anyone's bank credentials. A lead sees that, but only if the outcome is written down first.

A good **outcome statement** has this shape:

> **For** [who], **we want** [change in behaviour or result], **so that** [business or personal benefit]. **We will know it worked when** [measurable signal].

For Meera:

> For Meera and ten friends who forget to record daily spending, we want recording an expense to be quick enough that they do it on the day, so that at month-end they can see where their money went. We will know it worked when at least 7 of the 10 pilot users record expenses on at least 3 different days in each of 3 of their first 4 weeks.

**Where do the numbers come from?** I proposed them (for example "7 of 10"). A lead *proposes* a number, because clients often cannot invent one, and then asks the client to confirm or change it. A number nobody agreed to is not a success criterion.

**Three things a lead checks about a metric:**

1. **Is it measurable by the system?** "Users find it easy" is not. "Median time from opening the app to saved expense is under 10 seconds" is. But that metric only exists if the app *records* the time. So the metric creates a requirement: *the app must record usage events.* People forget this. **If you do not collect it from day one, you cannot analyse it later.** You, as a DBA, already know that you cannot query data that was never stored.
2. **Is it leading or lagging?** A *lagging* metric tells you the result late ("Do users still use it after three months?"). A *leading* metric gives you an early signal ("Did they log expenses in the first two weeks?"). With a two-month project you need leading metrics, because lagging ones arrive after the project ends.
3. **Can it be gamed or misleading?** "Number of downloads" is a *vanity metric*: it can look good while the product fails. "Active use by real users" is harder to fake.

Also add a **guardrail metric**, something that must *not* get worse while you chase the main metric. For example: "No incorrect totals reported by pilot users." You can easily raise engagement by making the app annoying with notifications; a guardrail stops you from "winning" the wrong way.

**A word on honesty with small numbers.** With ten users, no metric is statistically meaningful. Treat it as *directional evidence*, not proof. A lead says that out loud so nobody over-reads the results.

### 3.3 Step two: functional requirements

A **functional requirement** (FR) says **what the system must do**, in a way you can verify. A good FR:

- Starts from a capability, not a screen or a technology.
- Contains a clear actor and action: "A user can…", "The system records…".
- Is **traceable** to an outcome. If you cannot say which outcome it supports, it probably should not be in the MVP.
- Is **singular** (one requirement per statement), **unambiguous**, and **testable**.

Compare:

| Weak | Why it is weak | Stronger |
|---|---|---|
| "The app should have a nice add-expense screen with a dropdown." | Solution language (dropdown). "Nice" is untestable. | "A user can record an expense by entering an amount, choosing a category, and optionally a date and a note." |
| "Users can manage expenses." | "Manage" hides four different capabilities. | "A user can edit any field of an expense. A user can delete an expense." |
| "The app shows reports." | Which reports? For what period? | "A user can view, for any calendar month, the total spent in each category and the grand total." |

I use a quick **smell test** on words. If I see any of the following in a requirement, I stop and rewrite:

> *fast, easy, user-friendly, intuitive, robust, flexible, scalable, secure, modern, appropriate, as needed, etc., and/or, should be able to, support for, handle*

These words feel like requirements but have no measure. "Secure" is a goal. "Passwords are never stored in readable form and all traffic uses encrypted connections" is a requirement.

**Give every requirement an ID.** FR-1, FR-2, and so on. Then a story, a test, and a bug can all refer to the same number. This is called **traceability**, and it saves arguments.

### 3.4 Step three: non-functional requirements

Now the part many beginners skip. A **non-functional requirement (NFR)**, also called a **quality attribute**, says **how well** the system must behave, not what it does. This is where a DBA's experience is *most* valuable, so listen carefully.

Meera said: "super easy, super fast, super secure." Every one of those three words is an NFR in disguise, and each one costs money. Part of your job is to turn the adjectives into numbers and to explain the price.

Here are the categories I check on every project. Not all matter every time. The lead's skill is knowing which ones matter *here*:

| Category | Question to ask | Finance-app example |
|---|---|---|
| **Performance** | How fast, under what load? | Saving an expense responds in under 500 ms for 95% of requests with 50 concurrent users. |
| **Availability** | How much downtime is acceptable? | "The service is available 99.5% of each month" (about 3.6 hours of downtime). |
| **Data integrity** | What must never be wrong? | Monthly totals always equal the sum of the undeleted expenses. No expense is saved twice because of a double tap. |
| **Durability and recovery** | How much data may we lose and how fast must we recover? | **RPO** (Recovery Point Objective): lose at most 24 hours of data. **RTO** (Recovery Time Objective): service restored within 8 hours. |
| **Security** | What must be protected, from whom? | Passwords stored using a slow, salted hash. Each user can only read their own data. |
| **Privacy and compliance** | What laws or store rules apply? | User can delete their account and data. A privacy policy exists. |
| **Usability and accessibility** | Who must be able to use it? | Core flow completed in under 10 seconds by a first-time user. Text scales with system font size. |
| **Compatibility** | Which devices and versions? | iOS and Android versions covering at least 90% of the pilot users' devices. |
| **Observability** | How will we know when it breaks? | Errors are logged. Key user actions generate events for success metrics. |
| **Maintainability** | Can someone else take over? | Automated tests for critical flows. README that lets a new developer run the app in under an hour. |
| **Scalability** | How much growth should we survive? | Designed for 1,000 users without re-architecture; beyond that, not promised. |

**Notice the RPO and RTO row.** Those two terms are your home turf. You have probably set them for SQL Server databases. A client rarely knows them, so they say "never lose data." A lead replies: "Zero data loss costs real money, because it needs synchronous replication across locations. For ten users, losing up to a day in a disaster is realistic. Here is the price of each choice." That conversation is *requirements engineering*.

**Write NFRs so they are measurable.** A useful pattern is the **quality attribute scenario**:

> **Source** does **stimulus** under **conditions**, and the system responds with **response**, measured by **measure**.

Example: "A user on a mobile connection (source) taps Save twice quickly (stimulus) while on a slow network (condition). The system stores the expense once (response). Verified by: a test that sends the same request twice and checks that the database has exactly one row (measure)."

That example exposes a design consequence: to guarantee "exactly one row" the client should generate a unique ID for each expense and the server should reject duplicates. That is called **idempotency** (doing the same request twice has the same effect as doing it once). The NFR *forced* a design decision, and a unique constraint in PostgreSQL is the natural enforcer. That's how NFRs and your DBA skills connect.

**Every NFR has a cost, and some conflict.** Stronger security can reduce convenience. Higher availability raises hosting cost. Faster responses may need caching, which risks stale totals, and that conflicts with the data integrity requirement. A lead lays out these conflicts for the client rather than hiding them. *Right-size* the numbers to the context: promising 99.99% availability for ten pilot users is a waste of money, and a lead says so.

**What an inexperienced developer misses:** NFRs are invisible until they fail. Nobody writes "do not lose my data" as a feature, until the day data is lost.

**What an experienced lead investigates:** the *failure moments.* What happens when the network drops mid-save? When the phone clock is wrong? When the same expense is submitted twice? When a user deletes their account, does the data really go, including backups and logs, and for how long do backups retain it?

### 3.5 Step four: MVP scope

**MVP** stands for **Minimum Viable Product.** The most common misunderstanding is that an MVP is "a rough, half-built version." That is wrong. The better definition:

> An MVP is the **smallest product that lets us test our riskiest assumption** and **delivers the core outcome** to real users.

It must still work properly. A small product that works beats a large product that is unfinished.

**How a lead decides what is in.** I use a short process.

1. **List every candidate.** Everything that was requested or that you can think of. Do this *without judging*.
2. **Tie each candidate to an outcome.** If it does not clearly serve the outcome, it is a suspect.

3. **Ask the "foundation" question.** Some things are not features but *foundations* (sign-in, data ownership, backups, privacy). They don't excite anybody, but without them the product is unsafe.
4. **Sort using MoSCoW.** Must have, Should have, Could have, Won't have (this time). It is an established prioritisation technique. The trick is honesty: if everything is a Must, you have not prioritised.
5. **Draw the cut line** at the end of Musts, plus Shoulds that are cheap and valuable.
6. **Check that the Musts form one complete, usable journey.** This is called a **vertical slice**: a thin path through *every layer* (screen, API, database) that works end to end. The alternative, building all the screens first and then all the APIs, delivers nothing usable until the very end.
7. **For every "Won't," ask: do we need to protect the *data model* for it?** This is the "cheap insurance" idea from Lecture 1. We do not *build* shared households, but we do put `user_id` on every table so that we can add them later without a painful migration.

Let me apply this to Meera's list. I'll show you my reasoning for each, because the *reasoning* matters more than my verdict. (A different lead could draw some lines differently, and that is acceptable if the reasoning is written down.)

| # | Candidate | Verdict | Reasoning |
|---|---|---|---|
| 1 | Sign in with an account | **Must** | Data ownership and safety. Financial data on a phone only is lost with the phone. |
| 2 | Add expense (amount, category, date, note) | **Must** | The core of the outcome. |
| 3 | Edit and delete expense | **Must** | People make mistakes. Without correction, trust in the totals dies. |
| 4 | Fixed list of categories (about 8) | **Must** | Needed for the monthly breakdown. |
| 5 | Custom categories | **Won't (v1)** | Adds screens and rules. Fixed categories are enough to test the outcome. Design for it by storing categories as data, not hard-coded. |
| 6 | Monthly summary by category with totals | **Must** | Answers "where did my money go?", the heart of the problem. |
| 7 | List of expenses for a month, filter by category | **Should** | Helps users find a mistake. Cheap given 2 and 6 exist. |
| 8 | Daily reminder to log expenses (local notification) | **Should** | Directly attacks "I forget." Cheap, because a local notification needs no server. |
| 9 | Charts | **Could** | Totals in a list already answer the question. Charts are nice, not necessary for the outcome. |
| 10 | CSV export | **Should** | Data portability builds trust, is cheap, and it helps in disputes. |
| 11 | Account and data deletion | **Must** | Privacy duty. App stores generally require an in-app way to delete an account if the app lets you create one. *Verify current store rules before release.* |
| 12 | Usage analytics events (minimal) | **Must** | Without them we cannot measure the success metric. |

| # | Candidate | Verdict | Reasoning |
|---|---|---|---|
| 13 | Budgets and alerts | **Could → next release** | Valuable, but the pilot first needs to prove that logging works. Building budgets before people log expenses is building a roof without walls. |
| 14 | Auto bank import | **Won't (v1)** | Large cost, security and legal exposure, and the outcome can be reached more cheaply. Revisit if the pilot shows people still forget even with reminders. |
| 15 | Offline-first with later sync | **Won't (v1)** | Complex (conflict handling). For v1: the app requires a connection, but handles failure *safely* (no data loss, no duplicates). Cheap insurance: the app generates a unique ID per expense, which also makes offline possible later. |
| 16 | Shared households | **Won't (v1)** | Changes ownership everywhere. Insurance: `user_id` everywhere. |
| 17 | Multi-currency conversion | **Won't (v1)** | Insurance: store a currency code. |
| 18 | Receipt scanning | **Won't (v1)** | Heavy, uncertain value. |
| 19 | Dark mode | **Won't (v1)** | Cosmetic, two-way door, cheap to add later. |

**A subtle thing about line 15.** Meera did not ask for offline mode; Rahul did. I excluded it, but I did *not* ignore the underlying risk: bad networks. That became an NFR (no data loss, no duplicates on retry) and a recorded assumption ("users are mostly online when logging expenses"). This is the pattern: **an exclusion is not a dismissal. It is a decision with a reason, a protective measure, and a condition to revisit.**

**A subtle thing about line 13.** Anita's request for budgets is perfectly reasonable. We sequence it, not reject it. Notice that I justified it using *risk*: the riskiest assumption is "people will actually log expenses." Everything in the MVP exists to test that. Budgets only matter *if* that assumption holds.

**Scope creep and how the MVP defends itself.** *Scope creep* is the slow growth of scope through small "just one more thing" requests, each of which sounds reasonable. The defence is not stubbornness. It is (a) a written in-scope list, (b) a written exclusion list, and (c) a **change process**: any new request is evaluated for effect on time, cost, and risk, and the client decides what trades off. "Yes, and here is what that moves" is a professional response.

### 3.6 Step five: user stories

A **user story** is a short description of a capability from the user's point of view. The classic form:

> **As a** [type of user], **I want** [capability], **so that** [benefit].

The "so that" is the most valuable part. It forces a reason and links the story to an outcome. If you cannot write the "so that," the story may not belong.

A story is not a specification. It is a promise to have a conversation. The detail lives in the acceptance criteria.

**A good story passes the INVEST test.** INVEST is a widely used checklist:

- **I**ndependent: it can be built and delivered without needing another story first (as far as possible).
- **N**egotiable: details can be discussed; it is not a rigid order.
- **V**aluable: it gives value to a user or to the business.
- **E**stimable: the team can estimate it (if not, it is too vague, or you need a spike).
- **S**mall: it fits in a few days of work. Larger ones are called *epics* and should be split.
- **T**estable: you can tell whether it is done.

Example, weak versus strong:

| Weak | Problem | Strong |
|---|---|---|
| "As a user, I want expenses." | No capability, no benefit. | "As a pilot user, I want to record an expense in a few taps, so that I can do it right when I spend the money, before I forget." |
| "As a user, I want a database of my transactions." | A solution (database), not a capability. | "As a user, I want my expenses to still be there when I change phones, so that I don't lose my history." |
| "As a user, I want an analytics dashboard." | Too big (an epic), unclear benefit. | "As a user, I want to see how much I spent per category this month, so that I know where my money went." |

**Not all stories have a human "user."** Some stories serve the *business or operations*: "As the product owner, I want to see how many pilot users logged expenses each week, so that I know whether the pilot is succeeding." That story justifies the analytics events. Good leads write these, because otherwise important plumbing has no home in the backlog.

### 3.7 Step six: acceptance criteria

**Acceptance criteria (AC)** are the conditions a story must meet to be accepted as done. They answer: "How would a tester know this works?" Three principles:

1. **Testable.** A stranger could check them without asking you.
2. **Specific, with numbers.** Limits, formats, messages, boundaries.
3. **Cover more than the happy path.** For every story ask: what are the edge cases, the errors, the boundaries? Beginners write only "when everything goes right." Production lives in "when something doesn't."

A common format is **Given / When / Then** (from a practice called Behaviour-Driven Development):

> **Given** a starting situation, **when** something happens, **then** this is the observable result.

Here is a fully worked example, for the story *record an expense*. Watch how many questions appear that nobody asked in the meeting:

- **Given** I am signed in, **when** I enter an amount of 250.50, choose "Food," and tap Save, **then** the expense is saved with today's date and I see it in this month's total.
- **Given** the amount field is empty, zero, or negative, **when** I tap Save, **then** I see an error message and nothing is saved.
- **Given** I enter an amount with more than two decimal places (for example 10.999), **when** I tap Save, **then** the app does not silently round. It tells me the amount may have at most two decimals. *(Why? Silent rounding in a finance app is how you get a trust problem.)*
- **Given** an amount above 10,000,000.00, **when** I tap Save, **then** I see an error. *(The limit is a proposal that protects against typos like an extra zero.)*
- **Given** I choose a date in the future, **when** I tap Save, **then** I see a warning and I can confirm or correct. *(Decision to confirm with the client: allow or block?)*
- **Given** the connection is lost when I tap Save, **when** the app cannot reach the server, **then** I see a clear message, my input is still on the screen, and nothing is lost.
- **Given** I tap Save twice quickly, **then** exactly one expense is stored.
- **Given** a note of more than 200 characters, **then** I cannot enter more than 200.

Notice the **comments in parentheses.** Several criteria raised *decisions* ("allow future dates?"). A lead writes these down as **open questions** instead of guessing. Writing acceptance criteria is itself a discovery tool: it's cheaper to find these ambiguities in a document than in a bug report.

**Acceptance criteria versus Definition of Done.** They are different. AC are specific to one story ("amounts over 10 million are rejected"). The **Definition of Done (DoD)** is a team-wide

checklist that applies to *every* story ("code reviewed, tests passing, migration scripted, no new error in logs"). We will study DoD with quality gates in Module 5.

### 3.8 Step seven: assumptions, dependencies, open questions

Pull your assumption log from Lecture 1 into the PRD. Add two more lists:

- **Dependencies:** things outside your control that the project needs. Apple and Google developer accounts (they take time and cost money, and approvals can take days). A domain name. A privacy policy page. A hosting account. Someone has to own each of these and by when. Dependencies are a classic source of last-week surprises. App store review times, for example, are a known schedule risk.
- **Open questions:** decisions pending, each with an owner and a due date. "Allow future-dated expenses? Owner: Meera. Needed by: before the add-expense story starts."

A requirement that depends on an open question is marked as **provisional.** That's honest and useful.

### 3.9 Step eight: explicit exclusions

This is the most underrated section of any PRD. Silence is the enemy of scope. If something is not mentioned, each person will assume it is either included or excluded according to their own wishes. Then at the demo, somebody says "but of course it has reminders."

An **exclusion** (also called *out of scope* or a *non-goal*) states what we are deliberately not doing. There are three kinds, and a good lead tells them apart:

| Type | Meaning | Example |
|---|---|---|
| **Deferred** | Valuable, planned for later | Budgets and alerts |
| **Rejected** | We decided it doesn't belong | Receipt scanning |
| **Designed-for, not built** | We build nothing, but we protect the data model | Shared households (`user_id` everywhere) |

Each exclusion should carry **a reason** and, ideally, **a trigger to revisit**: "Bank import is excluded from v1 because of cost and compliance exposure. Revisit if at least 4 of 10 pilot users say they stopped logging because typing is too much." That sentence turns an argument into a condition, and it tells the client you listened.

### 3.10 The chain that holds everything together

If you do all this well, you can follow one thread from top to bottom:

```
Business outcome  →  Requirement (FR/NFR)  →  Story  →  Acceptance criteria  →
Test  →  Metric
"log on the day"     FR-2 record expense      US-2      "saved once if tapped
twice"   automated test   median time-to-log
```

If you can trace every requirement upward to an outcome, nothing is wasted. If you can trace every outcome downward to a test, nothing is unverified. If something has no upward link, it's probably scope creep. If something has no downward link, it's probably a wish.

---

## Part 4 — Practical demonstration: Meera's PRD, step by step

Now I'll do the whole process in sequence, the way I would on a real project. Then you'll see the *result*, the sample PRD, at the end of this part. Remember, the numbers and answers are for teaching, and in a real project you would confirm them with the client.

### Step 1 — Gather the inputs

Discovery notes, assumption log, and the wish-list from Part 2. I read them all together and underline anything that looks like a *solution* (for example "auto-import," "charts," "dark mode") versus a *need* (for example "never forget," "see where money goes").

### Step 2 — Write the outcome and the metrics

As in section 3.2:

- **Outcome:** recording an expense is quick enough that people do it the same day; at month-end they can see where money went.
- **Primary metric (leading):** at least 7 of 10 pilot users log expenses on at least 3 distinct days in each of 3 of the first 4 weeks.
- **Supporting metric:** median time from app open to saved expense under 10 seconds.
- **Guardrail:** zero reported incorrect totals; crash-free sessions of at least 99%.
- **Honesty note:** n = 10, so the evidence is directional.

**Consequence discovered:** we need usage events (`app_opened`, `expense_saved` with timestamps). That becomes FR and a story.

### Step 3 — Define users minimally

For a ten-person pilot, one **persona** is enough: "Pilot user: a salaried person in India with a smartphone, who spends small amounts daily on food, transport, and shopping, and currently tracks nothing or uses a notebook." Heavy persona workshops are for large products. Right-size.

**Step 4 — List and sort the candidates**

The MoSCoW table in section 3.5. The decisions that deserve the most thought were 13 (budgets), 14 (bank import), and 15 (offline), because they were *requested* and *excluded*. Those are the ones to explain to the client in plain words.

**Step 5 — Write FRs from the Musts and Shoulds**

Each FR must trace to the outcome:

- FR-1 Account (outcome: safe, owned data)
- FR-2 Record expense
- FR-3 Edit/delete expense
- FR-4 Categories
- FR-5 Monthly summary
- FR-6 Expense list
- FR-7 Reminder
- FR-8 Export
- FR-9 Account and data deletion
- FR-10 Usage events

**Step 6 — Write NFRs by walking the category table**

I go through the category table in section 3.4 and ask for each: does it matter here, and what number is *proportionate*? For ten users I choose 99.5% availability, RPO 24 hours, RTO 8 hours, p95 under 500 ms. I note that these are chosen **because the user base is small**, and I note what would change if it grows.

**Step 7 — Write stories and acceptance criteria**

For the Musts. I write the criteria and, as I do, I keep a list of every ambiguity I discover. I put them into open questions.

**Step 8 — Assumptions, dependencies, exclusions**

Assumptions come from the log. Dependencies from the release checklist (store accounts, privacy policy, domain, hosting). Exclusions from the MoSCoW "Won't" rows, each with a reason and a trigger.

**Step 9 — Smell test and read-through**

Last, I read the whole PRD **as the developer** ("could I build this without calling Meera?") and **as the tester** ("could I write a test for each line?"). Any "no" gets fixed.

### Weak approach versus stronger approach, side by side

| | Weak | Stronger |
|---|---|---|
| Start point | "Features list" copied from messages | Outcome and metric first |
| Language | "The app should be fast, secure, and look nice." | "Saving an expense responds in under 500 ms for 95% of requests." |
| Scope | Everything mentioned, all "high priority" | Explicit Must/Should/Could/Won't with reasons |
| Exclusions | None. ("We'll see.") | Written, with reasons and revisit triggers |
| Stories | "As a user I want budgets." | Valued, small, testable, with a "so that" |
| Acceptance | "It should work correctly." | Given/When/Then with boundaries and error cases |
| Unknowns | Hidden | Listed with owner and due date |

### The sample PRD section

Here is the result. I have kept it at a realistic length for an MVP and I've written it as the real document would look. I'm showing the whole thing in compact form so you can see how the parts connect.

markdown

```
# PRD: "Spendwise" (working name) — MVP Pilot
**Version:** 0.1 (Draft)   **Date:** 2026-10-12   **Author:** Tech Lead
**Status:** Draft — awaiting client review   **Client:** Meera

## 1. Purpose and background
Meera and about 10 friends forget to record daily spending (notebook today) and
cannot see at month-end where money went. This PRD defines a pilot mobile app
(iOS and Android) to test whether fast expense logging changes that behaviour.

## 2. Business outcome and success metrics
Outcome: Pilot users record expenses on the day they happen and can see where
their money went each month.

| Metric | Type | Target | How measured |
|---|---|---|---|
| Retained loggers | Primary (leading) | >= 7 of 10 users log on >= 3 distinct day
| Time to log | Supporting | Median <= 10 s from app open to saved expense | `app_
| Correct totals | Guardrail | 0 user-reported incorrect totals | Pilot feedback;
| Stability | Guardrail | >= 99% crash-free sessions | Crash reporting |
Note: n = 10, so results are directional, not statistically conclusive.

## 3. Users
Pilot user: salaried adult in India, smartphone, daily small spending, no or
paper-based tracking today. Product owner (Meera): wants weekly pilot visibility.

## 4. Scope — in (MVP)
Must: FR-1 to FR-5, FR-9, FR-10. Should: FR-6, FR-7, FR-8 (dropped first if
schedule is at risk).

## 5. Scope — explicitly out
| Item | Type | Reason | Revisit when |
|---|---|---|---|
| Bank account import | Deferred | High cost; security and legal exposure; outcome
| Offline-first with later sync | Deferred (design-for) | Sync conflicts add large
| Budgets and alerts | Deferred (next release) | Depends on logging habit being pr
| Shared households | Designed-for, not built | Ownership model change; `user_id`
| Multiple currencies / conversion | Designed-for, not built | Currency code store
| Custom categories | Deferred | Fixed list sufficient to test outcome | Users rep
| Charts | Deferred | Totals list answers the question | After pilot feedback |
| Receipt scanning | Rejected for now | Heavy, uncertain value | Not planned |
| Dark mode | Deferred | Cosmetic, cheap to add later | After pilot |
```

```
## 6. Functional requirements
| ID | Requirement | Priority | Outcome link |
|---|---|---|---|
| FR-1 | A user can create an account, sign in, and sign out. Each user can access
| FR-2 | A user can record an expense with amount (required), category (required),
| FR-3 | A user can edit any field of an expense and can delete an expense. Delete
| FR-4 | The app provides a fixed list of 8 categories (Food, Transport, Shopping,
| FR-5 | A user can view, for any calendar month, the total spent per category and
| FR-6 | A user can list the expenses of a month, filtered by category. | Should |
| FR-7 | A user can enable a daily reminder at a time they choose. | Should | Redu
| FR-8 | A user can export their expenses as a CSV file. | Should | Data portabili
| FR-9 | A user can delete their account, which removes their personal data (see N
| FR-10 | The system records usage events (app opened, expense saved) with timesta

## 7. Non-functional requirements
| ID | Category | Requirement | How verified |
|---|---|---|---|
| NFR-1 | Performance | Save-expense API responds in <= 500 ms at p95 with 50 conc
| NFR-2 | Usability | A first-time user records their first expense in <= 60 s wit
| NFR-3 | Availability | 99.5% monthly availability | Uptime monitor |
| NFR-4 | Data integrity | Monthly totals always equal the sum of undeleted expens
| NFR-5 | Reliability | If a save is retried or tapped twice, exactly one expense
| NFR-6 | Recoverability | RPO 24 h; RTO 8 h; restore tested once before launch |
| NFR-7 | Security | Passwords never stored in readable form; all traffic encrypte
| NFR-8 | Privacy | Account deletion removes personal data from live systems withi
| NFR-9 | Compatibility | Runs on iOS and Android versions used by all pilot devic
| NFR-10 | Observability | Server errors are logged and an alert reaches the lead

(Numbers are proposals sized for a 10-user pilot. Beyond ~1,000 users they must be

## 8. User stories and acceptance criteria

### US-2 Record an expense (FR-2)
As a pilot user, I want to record an expense in a few taps, so that I can do it
right when I spend the money, before I forget.
AC:
1. Given I am signed in, when I enter 250.50, choose Food, and tap Save, then the
   expense is saved with today's date and this month's Food total increases by 250
2. Given the amount is empty, zero, negative, or non-numeric, when I tap Save, the
   an error is shown and nothing is saved.
3. Given an amount with more than 2 decimals (e.g. 10.999), when I tap Save, then
```

```
   the amount is not rounded silently; I am told to use at most 2 decimals.
4. Given an amount above 10,000,000.00, then an error is shown.
5. Given I tap Save twice within one second, then exactly one expense exists.
6. Given the server is unreachable, when I tap Save, then I see a clear message,
   my entered data stays on screen, and nothing is stored twice on retry.
7. Given a note longer than 200 characters, then input stops at 200.
8. Given a future date, then I see a warning and can confirm or correct. [OPEN Q-1

### US-5 See where my money went (FR-5)
As a pilot user, I want to see how much I spent per category this month, so that
I know where my money went.
AC:
1. Given expenses exist this month, when I open the summary, then I see each
   category's total and the grand total, and totals equal the sum of undeleted exp
2. Given an expense is deleted, then the summary reflects it on the next view.
3. Given no expenses in a month, then I see an empty state with a prompt to add or
   and not zeros that look like an error.
4. Given an expense recorded at 23:59 on the last day of the month (my local time)
   then it counts in that month, not the next.
5. Given I choose a previous month, then I see that month's totals.
6. The summary loads in <= 1 s for up to 1,000 expenses in the month. [Proposal]

### US-10 Pilot visibility (FR-10) — business story
As the product owner, I want to see weekly counts of active pilot users, so that
I can tell whether the pilot is succeeding.
AC:
1. Given a user saves an expense, then an event with user ID and timestamp is stor
2. Events never contain amounts, categories, or notes.
3. A weekly report lists, per user, the number of distinct days with a saved exper

## 9. Assumptions
| ID | Assumption | Cost if wrong | Validation | Owner | Status |
|---|---|---|---|---|---|
| A1 | No bank import in v1 | High | Written confirmation | Meera | Open |
| A2 | INR only | High | Ask pilot users | Meera | Open |
| A3 | Each user sees only own data | High | Ask about couples/families | Meera |
| A4 | Users are mostly online when logging | Medium | Pilot week 1 observation |
| A5 | Pilot is about 10 users | Low | Meera | Meera | Confirmed |

## 10. Dependencies
| Item | Owner | Needed by |
|---|---|---|
```

```
| Apple Developer and Google Play accounts | Meera | Before build week 4 |
| Privacy policy page and support email | Meera and Lead | Before store submissior
| Hosting account | Lead | Before week 1 |
| App store review time (variable, allow buffer) | n/a | Plan for several days |

## 11. Open questions
| ID | Question | Owner | Needed by |
|---|---|---|---|
| Q-1 | Allow or block future-dated expenses? | Meera | Before US-2 starts |
| Q-2 | Fixed category list approved? | Meera | Before FR-4 |
| Q-3 | Preferred sign-in method (email + password vs one-time code)? | Meera and

## 12. Change log
| Version | Date | Change |
|---|---|---|
| 0.1 | 2026-10-12 | First draft |
```

**Walk through what I just showed you.** I want you to notice five things:

1. **Every requirement links upward** to an outcome and **downward** to a verification method.
2. **The numbers are proposals**, labelled, and sized to the context.
3. **The exclusion table is as long as the scope table.** That is on purpose. Half of scope control is saying "no" clearly.
4. **Acceptance criteria revealed open questions** (Q-1), instead of hiding them.
5. **Design consequences appear without being designed.** "Exactly one expense stored" implies idempotency. "Amounts exact" implies integer or `NUMERIC` money. "Month boundary in local time" implies time-zone handling. I did not write a design. I wrote requirements, and the design pressure showed up on its own. That is how Module 3 will begin: with these pressures.

---

## Part 5 — Documentation: what the lead records, and how much

### What goes in, and why

| Section | Why it exists | Detail level |
|---|---|---|
| Purpose and background | So a new reader understands why this exists | 3 to 6 sentences |
| Outcome and metrics | The compass and the judge | One table, each metric with target and measurement |
| Users | Who we serve | A paragraph per persona, no more |
| Scope in / out | The boundary | Tables with reasons; exclusions as detailed as inclusions |
| FRs and NFRs | The contract of behaviour | IDs, one requirement per row, a way to verify each |
| Stories and AC | The build-and-test detail | Stories for Musts first; Shoulds can wait until they are scheduled |
| Assumptions, dependencies, open questions | Visible risk | Owner and date on each |
| Change log | Accountability | One line per change |

### How much is enough?

A **rule of thumb:** write only what someone would otherwise have to ask you. If a section would not change what anyone builds or tests, cut it. For this MVP, the PRD above is realistic. For a bigger product, you split it: a short PRD plus separate feature specs.

### Practices that make a PRD useful

- **Version it** and keep it where the code lives or where the client can comment (for example a Markdown file in the repository, or a shared document). A PRD in somebody's email is a dead PRD.
- **Mark status:** Draft, In review, Approved. Get an explicit "approved" from the client in writing, even if it is a one-line email. In freelance work, that email is worth a lot.
- **Change control:** after approval, changes go through a short note: what changes, effect on time and cost, who approved. Nothing dramatic. A few lines in the change log is enough.

- **Review it with the client in conversation, not only by sending it.** People do not read documents carefully; they react to conversation. Walk through the metrics, the exclusions, and the open questions.
- **Write stories just-in-time.** You do not need detailed acceptance criteria for a Should feature that will be built in week 6. Detail decays as plans change.

**Anti-patterns**

- **The PRD written after the code,** to justify what was built.
- **Copy-pasted templates with empty sections.** A section nobody fills is noise.
- **Requirements without owners.** Someone must be able to answer questions about each one.
- **Acceptance criteria written by the developer alone,** after building. They turn into descriptions of what the code does, not what the user needs.

---

## Part 6 — Trade-offs and failure scenarios

I'll mark each point as **established practice** (widely agreed) or **context-dependent** (your judgment decides).

### 6.1 Too vague (established failure)

**Scenario.** The PRD says: "The app should be fast and secure." At the demo Meera says the summary takes four seconds on her older phone. You say "that's fast enough." She says it isn't. Both of you are right, because "fast" was never defined. **Consequence:** an argument, rework, and damaged trust.

### 6.2 Too detailed, too early (context-dependent)

**Scenario.** A 40-page PRD, with every screen and field, written before any user has touched anything. Half of it is wrong after the first pilot week. Detail is expensive to maintain, and false precision is worse than honest uncertainty. **Balance:** more detail where being wrong is costly (money, data, compliance), less detail on things that you can easily change (colours, wording).

### 6.3 Solutions disguised as requirements (established)

"The app must use a dropdown for categories" locks the design. The *requirement* is "user selects one of the categories in as few taps as possible." A tester can test that, and the designer is free to find a faster interaction, for example showing the categories as large tappable chips.

### 6.4 NFRs that are wishes (established)

"Highly available, scalable, secure." Without numbers they cannot be built, tested, or priced. Another subtle failure: **NFRs invented without cost.** If the client says "99.99%" and nobody

explains that it may multiply hosting cost and complexity, the project will either overspend or silently fail the promise.

### 6.5 MVP too big, or too small (context-dependent)

- **Too big:** "MVP" with bank sync, budgets, and charts. It takes six months, and you learn nothing in the meantime.
- **Too small:** an MVP so thin that users cannot complete the core journey (for example you can add an expense but you cannot fix a mistake). Users leave, and you wrongly conclude that nobody wants the product. **The MVP must be small but whole.** A phrase some teams use is *"minimum lovable product"*: small, but good enough that someone would genuinely use it.

### 6.6 Everything is a Must (established)

If the client insists every item is a Must, apply a **forced trade**: "If we have to cut one item to protect the date, which one?" The answer reveals the real priority.

### 6.7 Stakeholder conflict (context-dependent)

Rahul wants bank import; Anita wants budgets; Meera wants charts. A lead does not "average" the wishes. The lead returns to the *outcome* and the *decision-maker*. In our case, the client who pays and owns the product, Meera, decides, informed by the outcome and the evidence. In an organisation, the product owner decides. Decide *who decides* early, in writing.

### 6.8 The PRD that never gets updated (established)

Scope drifts, the document does not, and a month later nobody trusts it. Fix: update the PRD whenever scope changes, even by one line, and keep the change log. If you cannot keep it current, make it shorter.

### 6.9 Acceptance criteria written after the fact (established)

Criteria written after building describe the code. Criteria written before reveal missing decisions while they are cheap to answer.

### 6.10 Alternatives to a classic PRD (context-dependent)

- **One-page lean brief:** problem, outcome, scope in/out, risks. Often enough for a two-week task.
- **User story mapping:** arranging stories along the user's journey to find the thinnest slice that makes the journey work. Good for finding an MVP.
- **Jobs To Be Done:** frames needs as "when I am in situation X, I want to do Y, so that Z." Good for exploring the *why*.
- **Appetite-based pitches (as in the Shape Up method):** fix the time you are willing to spend and shape the scope to fit it. Useful when the deadline is firmer than the scope.

None is the one correct process. For a freelance MVP I prefer a short PRD plus story map, because the client gets a document to approve and I get a build order.

---

## Part 7 — Hands-on exercise

Here is a task for you to attempt on your own. I will then show you what a typical attempt looks like and how I would review it. **If you want to benefit from the exercise, write your own answers before reading Part 8.**

### The scenario

The pilot ran for four weeks (invented data). Six of the ten users met the retention target. Meera is happy. She forwards you two messages:

> **Anita:** "I'd love a monthly budget for each category, like 5,000 for Food, and I want a warning when I'm close and when I go over. Rahul says it should *block* me from spending, ha."
> **Meera:** "Warn me, but please don't annoy me. And I'd like it for the next release in about three weeks."

Your candidate list for **Release 1.1**:

1. Monthly limit per category
2. One overall monthly limit
3. Alert when 80% of a limit is used
4. Alert when 100% is reached
5. Remote push notifications for alerts
6. Rollover of unspent budget to next month
7. Shared budgets between two people
8. Weekly or salary-cycle budget periods
9. Forecast: "at this pace you will exceed by the 22nd"
10. Blocking spending when the limit is reached

### Tasks (about 60 to 75 minutes)

1. **Outcome and metrics.** Write an outcome statement. Add one leading metric and one guardrail metric, each with a target and a way to measure it.
2. **MVP cut.** Sort the ten candidates into Must / Should / Could / Won't for Release 1.1, with a one-line reason each.
3. **Functional requirements.** Write four to six FRs for the in-scope items. No solution words.

4. **Non-functional requirements.** Write at least three measurable NFRs, including one about correctness and one about privacy or notification behaviour.
5. **Stories and acceptance criteria.** Write two user stories, each with at least six acceptance criteria. Use Given/When/Then for at least three of them, and include at least two edge cases or boundaries.
6. **Assumptions.** Write at least three, each with a cost-if-wrong and a way to validate.
7. **Exclusions.** Write at least four, each with a reason and a revisit trigger. Mark each as deferred, rejected, or designed-for.

---

## Part 8 — Review of a typical attempt

Since this lecture is one-way, I'll review the sort of answer a learner at your level commonly writes. It contains realistic mistakes. Notice that I give hints first, then the correction, as I would in a live session.

### 8.1 A typical first attempt (call the learner "Sam")

> **1. Outcome:** Build a budget feature so users can set budgets. Metric: users like it.
> **2. MVP:** Must: all ten items. Reason: Anita and Meera asked for them.
> **3. FRs:**
> - FR-1: The app shall have a Budget screen with a dropdown for the category and a slider for the amount, built with Redux.
> - FR-2: The app shall send push notifications using Firebase.
> - FR-3: The system should be able to handle budgets etc.
>
> **4. NFRs:** The app should be fast and secure and easy to use.
> **5. Story:** As a user, I want budgets. AC: Budget works correctly. Alerts show properly.
> **6. Assumptions:** Users will like it.
> **7. Exclusions:** None. We will add the rest later.

### 8.2 The review

**What Sam did well.**

- Sam picked the right feature to focus on and correctly noticed that notifications are a central part of it. That is the right instinct.
- Sam attempted a story and tried to write acceptance criteria. Plenty of beginners skip them.
- Sam named a real technology (Firebase) that is a realistic option for push notifications. The *knowledge* is good. The *placement* is wrong, as I'll explain.

**Gap 1: the outcome is an output.** "Build a budget feature" is a task, not an outcome. "Users like it" cannot be measured. *Hint:* Ask: what is different in a user's life *after* budgets exist? When

would they learn about overspending today, and when would they like to learn about it? *Correction:* "Users learn about approaching overspend while they can still act, rather than at month-end." A leading metric: "At least 5 of 10 pilot users create at least one budget within 14 days of release." A guardrail that directly honours Meera's "don't annoy me": "Alert notifications are never shown more than once per threshold per category per month, and fewer than 20% of users turn alerts off."

**Gap 2: everything is a Must.** No prioritisation means no decision. Items 7 (shared budgets), 10 (blocking), and 6 (rollover) are especially problematic. *Hint:* Which items change the *data ownership model*? Which are *technically impossible* for an app that only records spending? Which need historical data we do not yet have? *Correction:* Blocking spending is not feasible for an expense *recorder*, because the app does not control payments. Shared budgets change ownership. Forecasting needs months of data. All three are out. The Musts are small: per-category limit, progress view, 80% and 100% alerts.

**Gap 3: requirements contain solutions.** "Dropdown," "slider," "Redux," "Firebase" are design and technology choices. A tester cannot tell whether "built with Redux" is a requirement for the *user*. *Hint:* Rewrite each as "A user can… / The system does…" and delete every library name. *Correction:* "A user can set, change, and remove a monthly spending limit for any category."

**Gap 4: FR-3 is untestable.** "Should be able to handle budgets etc." contains three smell words: "handle," "should be able to," and "etc." *Hint:* What exactly happens, and to what, when? *Correction:* "When saving, editing, or deleting an expense changes a category's spending so that it reaches or exceeds 80% or 100% of its limit for the first time that month, the user is shown an alert."

**Gap 5: the NFRs are adjectives.** "Fast, secure, easy" cannot be verified. Also, the review missed the *interesting* NFRs: correctness of budget totals, month boundaries in the user's time zone, privacy of what appears on a lock screen, and duplicate alerts. *Hint:* What would be embarrassing for the client? An alert that says "you used 85%" when the total is really 60%. *Correction:* "Budget status always equals the sum of undeleted expenses for the category in the calendar month in the user's time zone; verified by an automated reconciliation test." And: "Alert text does not display amounts on a locked screen unless the user opts in."

**Gap 6: story with no benefit and no testable criteria.** "As a user, I want budgets" has no "so that." "Works correctly" is not an acceptance criterion. *Hint:* Walk through the boundary values. What if spending is *exactly* 80%? What if one expense crosses both thresholds at once? What if an expense is deleted afterwards? What if a new month begins? *Correction:* see the reference solution below.

**Gap 7: the assumption is a hope.** "Users will like it" has no validation and no cost-if-wrong. A useful assumption is something that, if false, changes the plan. Example: "In-app alerts are

enough, because users open the app when they log an expense." If that is wrong, push notifications become a Must.

**Gap 8: no exclusions.** "We will add the rest later" is the exact silence I warned about. In three weeks somebody will say "I thought rollover was included."

**Overall verdict.** Sam understood *what* the feature is but wrote the document as a wish-list. The fix is a change of habit, not more knowledge: **climb to the outcome first, then write only things you can verify.** I would ask Sam to resubmit tasks 1, 2, and 5 and then compare against the reference below.

---

## Part 8B — Reference solution to the exercise

Try not to read this until you have made your own attempt.

### 1. Outcome and metrics

> **Outcome.** For pilot users who want to control spending, we want them to be warned as they approach and reach a monthly limit per category, so that they can change behaviour *before* month-end.

| Metric | Type | Target | Measure |
|---|---|---|---|
| Budget adoption | Leading | At least 5 of 10 pilot users create at least one budget within 14 days of release | `budget_created` events |
| Alert annoyance | Guardrail | No more than one alert per threshold per category per month; fewer than 20% of users disable alerts in the first month | Alert event counts; settings events |
| Correct status | Guardrail | 0 user-reported incorrect budget totals | Feedback and reconciliation test |

## 2. MVP cut

| # | Candidate | Verdict | Reason |
|---|---|---|---|
| 1 | Monthly limit per category | **Must** | What Anita asked for, and the core of the outcome. |
| 2 | Overall monthly limit | **Should** | Useful, simple, but separate from category limits and not requested precisely. |
| 3 | 80% alert | **Must** | "Close to the limit" is exactly the early warning. |
| 4 | 100% alert | **Must** | "Gone over." |
| 5 | Remote push notifications | **Could** | Needs push infrastructure and permissions. In-app alert at the moment of logging reaches the user when they perform the behaviour. Revisit if users do not open the app. |
| 6 | Rollover | **Won't** | Adds money-rule complexity (what rolls over after edits?). |
| 7 | Shared budgets | **Won't** | Changes ownership model; same reason as shared households. |
| 8 | Weekly or salary-cycle periods | **Won't (1.1)** | Real need possible (many people are paid mid-month), but it changes how every total is computed. Validate demand first. |
| 9 | Forecast | **Won't** | Needs several months of data and a sound method. |
| 10 | Blocking spending | **Won't (rejected)** | The app records spending; it does not control payments. Cannot deliver, and the client asked for warnings. |

## 3. Functional requirements

- **FR-B1.** A user can set, change, and remove a monthly spending limit for any category. One limit per category. The amount must be greater than zero.
- **FR-B2.** For each category with a limit, the monthly summary shows amount spent, the limit, the amount remaining, and the percentage used.
- **FR-B3.** When saving, editing, or deleting an expense causes a category's spending to reach or exceed 80% or 100% of its limit for the first time in a calendar month, the user is shown an alert.
- **FR-B4.** Each threshold alert is shown at most once per category per calendar month. Alerts are not withdrawn if spending later drops.
- **FR-B5.** A budget month is the calendar month in the user's time zone. Spending resets on the first day of the month, and unspent amounts do not carry over.

- **FR-B6 (Should).** A user can set one overall monthly limit across all categories, with the same alerts.

**4. Non-functional requirements**

- **NFR-B1 Correctness.** Budget status always equals the sum of undeleted expenses in the category for that month, using the user's time zone for month boundaries. Verified by automated reconciliation tests, including expenses at 23:59 and 00:00 on month boundaries.
- **NFR-B2 Performance.** The response that saves an expense also returns updated budget status, with p95 at most 500 ms at 50 concurrent users (illustrative figure).
- **NFR-B3 Privacy.** Alert text shown outside the app (if push is added later) does not display amounts by default. Budget data is included in the user's CSV export and removed by account deletion.
- **NFR-B4 Observability.** Alert-shown events are recorded (without amounts) so that the annoyance guardrail can be measured.
- **NFR-B5 Reliability.** Retrying a save does not create duplicate alerts or duplicate expenses.

**5. Stories and acceptance criteria**

**US-B1 Set a limit.** As a pilot user, I want to set a monthly limit for a category, so that I know how much I can still spend there.

1. **Given** no limit for Food, **when** I set 5,000.00, **then** Food shows limit 5,000.00, spent so far this month, and the remaining amount.
2. **Given** an existing limit, **when** I change it to 6,000.00, **then** remaining and percentage update immediately, and no alert is shown merely because I changed the limit.
3. **Given** I enter zero, a negative, or a non-numeric value, **then** an error is shown and nothing is saved.
4. **Given** more than two decimal places or a value over 10,000,000.00, **then** an error is shown.
5. **Given** the new limit is lower than what I have already spent, **when** I save, **then** it is accepted and the category shows "over limit." [OPEN Q-B1: should this also show an alert?]
6. **Given** I remove a limit, **then** no recorded expense is changed or deleted, and the category no longer shows progress.
7. **Given** I tap Save twice quickly, **then** exactly one limit exists for the category.

**US-B2 Get an early warning.** As a pilot user, I want to be told when I am close to or over a limit, so that I can change my spending before month-end.

1. **Given** Food limit 5,000 and spent 3,900, **when** I save an expense of 200 (total 4,100, which is 82%), **then** I see an alert "Food: 82% of limit used" right after saving.
2. **Given** the spending is **exactly** 4,000 (80%), **then** the alert is shown, because the threshold is "reach or exceed."
3. **Given** the 80% alert has already been shown this month for Food, **when** another expense keeps it between 80% and 100%, **then** no new alert is shown.
4. **Given** one expense takes Food from 70% to 105%, **then** one alert is shown for the highest threshold reached (100%), not two. [OPEN Q-B2: confirm with Meera.]
5. **Given** I delete an expense and spending drops below 80%, **then** no alert is shown. **And given** it later rises past 80% in the same month, **then** no new 80% alert is shown.
6. **Given** a new calendar month in my time zone begins, **then** spending counters reset and alerts can be shown again.
7. **Given** I add an expense dated in a previous month, **then** it affects that month only and shows no alert for the current month.
8. **Given** I retry a save after a network error, **then** at most one alert is shown for it.

### 6. Assumptions

| ID | Assumption | Cost if wrong | Validation |
|---|---|---|---|
| A-B1 | In-app alerts are enough because users open the app each time they log an expense | **Medium to high**: push notifications become a Must, adding infrastructure and permission handling | Pilot observation; ask 5 users after two weeks |
| A-B2 | 80% and 100% are suitable thresholds | Low | Ask pilot users; thresholds could later be configurable |
| A-B3 | Calendar-month budgets match how pilot users think (not salary-cycle) | **Medium to high**: changes how all totals are computed | Ask all 10 users one question: "When does your month start?" |
| A-B4 | Single currency (INR) remains true | High | Ask pilot users |

### 7. Exclusions

| Item | Type | Reason | Revisit when |
|---|---|---|---|
| Blocking spending | Rejected | The app records; it does not control payments | Never, unless the product integrates payments |
| Rollover | Deferred | Money-rule complexity; no demonstrated need | At least 3 of 10 users ask |
| Shared budgets | Designed-for, not built | Needs a shared-household ownership model | Shared-households feature is approved |
| Weekly or salary-cycle periods | Deferred | Changes all totals; validate demand with A-B3 | At least 3 of 10 users have a non-calendar cycle |
| Forecasting | Deferred | Needs months of history and a sound method | After 3 months of pilot data |
| Remote push alerts | Deferred | Infrastructure and permission cost; A-B1 untested | A-B1 proves false |

**What to notice in the reference.** The scope shrank, but the *outcome* is still delivered. The edge cases (exactly 80%, a double-threshold jump, deletion, retry, month boundaries, back-dated expense) all came from asking "what happens when…?" about money and time. Those are the same kinds of cases a DBA thinks about for integrity: boundaries, duplicates, and what happens on delete.

---

## Part 9 — Recall test, with answers

Cover the answers and try to respond first. If you can answer without looking at the lecture, you have understood the ideas rather than memorised the template.

**Q1. Why is "Can it show nice charts?" not a requirement?** *Answer:* It is a solution request. The requirement is the underlying need, for example "users can understand where their money goes without calculating." Charts are one way to meet it. A totals list may meet it as well, more cheaply.

**Q2. What is the difference between a functional and a non-functional requirement? Give a finance-app example of each.** *Answer:* A functional requirement says what the system does ("a user can record an expense"). A non-functional requirement says how well ("saving responds in 500 ms for 95% of requests," or "a double tap stores one expense").

**Q3. Why does a good success metric sometimes create a requirement?** *Answer:* If the metric needs data that the system must collect (for example time to log), then collecting it becomes a requirement (usage events). Data not collected from day one cannot be analysed later.

**Q4. What is an MVP, and what is the most common misunderstanding about it?** *Answer:* The smallest product that delivers the core outcome and tests the riskiest assumption. The common mistake is thinking it means a rough or unfinished product. It should be small but whole and working.

**Q5. A client says "bank import is a must." What do you do?** *Answer:* Go back to the outcome ("people forget to log"). Ask what alternatives could reach it more cheaply (fast entry and reminders). Explain the cost, security, and compliance exposure. Offer the forced trade ("what do we cut to include it?"). If it is deferred, write the reason and a revisit trigger, and protect the data model where cheap.

**Q6. What is wrong with this acceptance criterion: "The summary works correctly"?** *Answer:* It is not testable. A stranger could not tell whether it passed. A good criterion states a situation, an action, and an observable result with specific values, and includes boundaries and errors.

**Q7. Name the three kinds of exclusions and give an example of each from today.** *Answer:* Deferred (budgets in the MVP), rejected (receipt scanning, blocking spending), designed-for but not built (shared households via `user_id`; currency via `currency_code`).

**Q8. Why does an exclusion need a revisit trigger?** *Answer:* It turns "no" into "not yet, and here is the evidence that would change my mind." It reduces arguments, shows the client they were heard, and prevents permanent rejection of something that later proves necessary.

**Q9. Why is "exactly one expense is stored when I tap Save twice" a database and design concern, not just a screen concern?** *Answer:* It requires idempotency: each expense carries a unique ID generated once, and a unique constraint in the database rejects duplicates. The requirement drives the schema and API design.

**Q10 (challenge). A client asks for 99.99% availability for a ten-user pilot. How do you respond as a lead?** *Answer:* Explain what it means (about 4 minutes of downtime per month), what it costs (redundancy across zones, failover, on-call), and that the benefit for ten pilot users is small. Propose 99.5% for the pilot with a note about when to revisit (user growth or paid customers). Record the decision as a proposal and get agreement.

---

## Part 10 — Completion checklist

Tick these honestly. If you cannot tick one, revisit the section noted.

- [ ] I can turn a feature request into an **outcome statement** with a measurable target (3.2).

- [ ] I can tell a **leading** metric from a **lagging** one, and I know what a **guardrail** metric protects (3.2).
- [ ] I can write a **functional requirement** without solution language, and I can apply the **smell-word test** (3.3).
- [ ] I can turn adjectives like "fast" and "secure" into **measurable NFRs** and I can explain **RPO** and **RTO** (3.4).
- [ ] I understand how an NFR can **force a design decision**, for example idempotency or exact money storage (3.4, 4).
- [ ] I can define an **MVP** correctly and sort candidates using **MoSCoW** with reasons (3.5).
- [ ] I can explain the difference between a **vertical slice** and building layer by layer (3.5).
- [ ] I can write a user story that passes **INVEST** (3.6).
- [ ] I can write **Given/When/Then** criteria that include boundaries, errors, and duplicates (3.7).
- [ ] I can distinguish **acceptance criteria** from **Definition of Done** (3.7).
- [ ] I can list **assumptions, dependencies, and open questions** with owners and dates (3.8).
- [ ] I can write **exclusions** with a reason and a revisit trigger, and I know the three types (3.9).
- [ ] I can trace a requirement **up to an outcome** and **down to a test** (3.10).
- [ ] I understand that PRD depth is **context-dependent**: scale it to the cost of being wrong (6).

---

## Closing: what you should understand before moving on

Before the next lecture, make sure you can say, in your own words, these five things:

1. **A PRD defines what and why, not how.** Solutions belong in the technical design.
2. **Outcomes come first.** Every requirement should trace to an outcome, and every outcome should be verifiable.
3. **NFRs carry hidden cost and design pressure.** Turn adjectives into numbers, and explain the price.
4. **An MVP is small but whole.** Scope is defended by written exclusions with reasons and revisit triggers, not by stubbornness.
5. **Acceptance criteria are a discovery tool.** Writing them early exposes the decisions that nobody made yet.

This is the end of Module 2, Lecture 1. I'm stopping here, as instructed.